

Image Analysis Project - Basic Crack Detection

Nagaraju Saggu

1. Introduction

Automated crack detection has become an essential tool for civil engineers, transportation, and building safety. Structural cracks pose a significant threat to the integrity of infrastructure, including bridges, roads, and buildings. Traditional manual inspection is time-consuming and prone to human error, leading to maintenance delays. Automated systems provide a faster, more accurate alternative by leveraging advanced image processing technologies.

The structural integrity of buildings and infrastructure is crucial for their safe operation. Historically, cracks have developed due to stress over time. Traditional crack inspection was time-consuming and prone to error, relying on human observation.

This project aims to detect cracks using classical image processing without relying on neural networks. The developed crack detection system will streamline inspections effectively, focusing on cracks in concrete walls within buildings.

2. Task A – Data Engineering

Data collection was a delicate process, especially considering the impact of environmental factors. For example, varying levels of sunlight and shade could significantly alter the lighting conditions, making crack areas difficult to detect. As a result, it was crucial to carefully annotate the data, ensuring that cracks were accurately labeled even under suboptimal conditions.

To address these challenges, various data augmentation techniques were employed to simulate different conditions and enhance the model's resilience. Transformations such as rotation, flipping, and contrast adjustments were utilized to mitigate potential issues related to lighting and perspective in real-world scenarios.

2.1 Data Acquisition

The data for the project was gathered by capturing images of cracks in urban walls. Photos were taken in public areas whenever feasible, and private property was generally excluded. A total of 10 photographs were captured using a camera phone, with each image accompanied by a detailed description of its location.

2.2 Data Annotation

The images have been annotated using pixel-level segmentation, where crack regions are assigned a value of 255 and non-crack regions are assigned a value of 0. Annotation tools such as GIMP and CVAT were utilized for this purpose.

Tools Used: For manual pixel-level annotation, GIMP and CVAT were used, enabling accurate crack labeling.

Annotation Process: To ensure consistency across the dataset, cracks were meticulously outlined and labeled in images.

Challenges: Two significant obstacles that needed to be addressed were changing lighting conditions and ensuring consistency in annotations.

Precision is important because it helps the model learn intricate patterns by identifying thin or irregular cracks. Pixel-level annotation is essential for this.

Impact on Model: More precise annotation improves segmentation and overall performance in crack detection tasks by directly influencing the quality of the model's predictions.

2.3 Data Splitting

In order to thoroughly evaluate the effectiveness of the crack detection system, the dataset was divided into two sets: 80% for training the model and 20% for testing its performance. This division was implemented to guarantee an impartial assessment of the model's capabilities when faced with new, unseen data.

Training Set (80%): This is the model's training set, which teaches it how to find patterns in the data.

Testing Set (20%): Set aside to assess how well the model performs with untested data.

Prevents Overfitting: Aids in creating a robust model that adapts well to new data.

Equitable Performance Assessment: Examining distinct data sets guarantees the model's capacity to function in actual situations.

2.4 Data Augmentation

The process of data augmentation involves applying various techniques such as rotation, flipping, and adjusting contrast to introduce additional variations to the dataset. This is done to prevent overfitting of the training models. While data augmentation is not mandatory, it was implemented in the dataset to enhance the diversity of the data and improve the robustness of the models during training.

The model can be strengthened by adding more variations using additional techniques like cropping, zooming, and shearing. Data augmentation makes sure that the model picks up on crack detection even when conditions or image perspective changes. While it's not always necessary, augmentation comes in handy when the original dataset is small because it effectively multiplies the number of training examples, which improves performance.

Rotation and Flipping: Create the appearance of fractures at various angles and orientations.

Contrast Adjustment: Assists in teaching the model to identify fractures in different lighting scenarios.

Prevents Overfitting: By adding variability, augmented data lessens the reliance of the model on the particular characteristics of the training set.

Boosts Robustness: The model's ability to generalize to a wider range of real-world situations is improved.

Important for Small Datasets: By extending small datasets, augmentation enhances the performance and learning process of the model.

2.5 Dataset Statistics

The initial analysis of the 10-image dataset involved calculating the distribution of pixel intensities as well as determining the ratios of cracked to non-cracked areas. This allowed for a more comprehensive understanding of the characteristics of the dataset.

Pixel Intensity Distribution: Understanding the brightness and contrast of an image can be achieved by examining the range and frequency of pixel values.

Crack-to-Non-Crack Ratio: Determining the ratio of cracked to non-crack regions aids in determining the equilibrium of the dataset.

Imbalance in the dataset: To enhance the model's learning, the dataset may need to be weighted or augmented if it is skewed toward non-crack regions.

Notifies Preprocessing: Assists in ascertaining if extra preprocessing actions (like normalization) are required to standardize picture data prior to training.

3. Task B – Crack Segmentation

The use of thresholding was crucial for effectively segmenting crack regions from the background. A simple global threshold was not suitable due to non-uniform lighting conditions in the images; therefore, adaptive thresholding was necessary. This involved adapting the threshold for small regions to achieve accurate segmentation. Subsequent smoothing was carried out using morphological operations. For example, erosion was employed to eliminate small noise particles, while dilation restored continuity to crack regions and bolstered their structural integrity for further analysis.

3.1 Thresholding Method

Pixel Intensity Thresholding: This technique was used to distinguish cracks from the background because cracks frequently show up as dark lines against lighter backgrounds. The theory is that, in comparison to non-crack regions, pixels inside cracks have lower intensity values (darker).

Adaptive Thresholding: Images with different lighting conditions would not benefit from the use of a global threshold, which sets a single intensity value for the entire image. Adaptive

thresholding was therefore used. By using this technique, the algorithm can take into account varying lighting conditions throughout the image by computing a local threshold for smaller regions of the image. This guarantees accurate crack detection even in areas with uneven lighting.

In this task, pixel intensity thresholding for the detection of cracks was implemented because cracks are mostly black lines appearing against a white background and this method also includes adaptive thresholding to adapt to varying illumination conditions.

3.2 Morphological Operations

Following thresholding, morphological operations are used to clean up the output because the segmentation results might include noise or broken crack regions.

Erosion: Through the "shrinking" of objects in the image, such as cracks, this operation eliminates tiny noise particles. Erosion can be used to remove isolated, tiny areas that don't indicate fractures.

Dilation: Cracks may thin out or fracture following erosion. Dilation makes crack regions more connected and simpler to study by helping to "expand" and restore their continuity.

The outputs of the segmentation step were processed using morphological operations such as erosion and dilation to smooth the results and remove background noise. This was done to ensure that the identified cracks appeared continuous and well-defined.

3.3 Connected Component Analysis

Connected component analysis was utilized to identify and isolate individual areas of cracks into distinct groups for subsequent analysis. It was assumed that each identified region was independent and separate from the others.

This entails treating every crack or crack segment as a separate entity for the purpose of crack detection, allowing for individual analysis. This is essential for obtaining characteristics (such as length or form) for every fracture that is found.

3.4 Feature Engineering

The features such as length, circularity, and pixel intensity were extracted for each segmented crack region. These features played a crucial role in determining whether a region belonged to a crack or non-crack area during the classification process.

Length: The total count of each crack's connected pixels.

Circularity: This characteristic aids in separating areas with or without cracks from those with cracks. Long, narrow forms characterize low circularity in cracks, while higher circularity can be seen in non-crack regions.

Pixel Intensity: To distinguish crack regions from background noise, one uses the intensity values of the pixels inside the crack region.

3.5 Classifier Implementation

A rule-based classifier was developed to classify regions by analyzing the thresholds of their features. However, after conducting experiments, it was found that more advanced classifiers like Support Vector Machines (SVMs) and decision trees yielded improved results.

First, each region was classified as either a crack or non-crack using a rule-based classifier. The extracted features (like length and circularity) were compared to predetermined thresholds in order for the classifier to function. For instance, a region would be categorized as non-crack if its circularity was high.

Subsequent experiments revealed that machine learning classifiers—like decision trees and Support Vector Machines (SVMs)—performed better. Based on the features that were extracted, these classifiers might be able to identify more intricate patterns in the data and distinguish cracks from non-cracks more successfully.

4. Task C - Crack Analytics

The thinning process was of prime importance while transforming crack regions to their skeletal representation. The iterative boundary erosion along with the central line preservation resulted in the reduction of cracks to its core as well as facilitated easy calculation of the precise length of detected cracks. That is the reason why this method is called morphological thinning and is primarily efficient for line-like features like cracks

4.1 Performance Metrics

The crack detection system's performance is evaluated using the Intersection over Union (IoU) metric. This metric calculates the overlap between the predicted crack regions and the ground truth crack regions in images, providing a measure of accuracy

for the model. The IoU score is computed for each image to assess the model's performance comprehensively.

$$IoU = \frac{\textit{Area of Union}}{\textit{Area of Overlap}}$$

4.2 Thin Cracks

The thinning procedure thins the cracks' thickness to a width of a single pixel, making it simpler to analyze the fractures and determine their length, direction, and connectivity. This skeletal form aids in post-processing procedures like figuring out the overall crack length or crack branching patterns by preserving the fundamental geometric properties of the cracks without removing the majority of the original segmentation.

4.3 Length Computation of Cracks

When identifying cracks, it's important to measure the length of each crack concerning the centerlines to accurately determine the size of the crack.

4.4 Discussion of Results

The initial detection showed promise for identifying basic cracks, but it struggled with more intricate patterns. While the method is relatively straightforward to apply, it needs further development through the incorporation of machine learning models.

S.No	Image Number	IoU	Total Crack Length (pixels)
1	Image 1	0.26	503.51
2	Image 2	0.44	525.61
3	Image 3	0.39	1729.89
4	Image 4	0.01	7409.04
5	Image 5	0.58	1102.85
6	Image 6	0.41	410.97
7	Image 7	0.05	995.07
8	Image 8	0.43	1551.47
9	Image 9	0.34	824.91
10	Image 10	0.59	2852.54

Totals:

Mean IoU on all images: 0.35

Mean Crack Length on all images: 1790.59 pixels

Explanation:

The total crack length for each image is the sum of the individual crack lengths found in that image.

The IoU and total crack length provide insights into the performance of the segmentation process for each image and the extent of cracks.

Total images: 10

Total pixels: 6415759

Crack pixels: 95066 (1.48%)

SVM Classifier Accuracy: 60.00%

Drawbacks while implementing

The low Intersection over Union (IoU) score for Images 4 and 7 is a major flaw in the thinning process. The overlap between the actual ground truth cracks and the predicted crack regions (derived from the thinning results) is measured by the IoU metric. The IoU is noticeably low in these particular images, suggesting that the thinning algorithm is unable to fully capture the crack structures.

Several factors contribute to this issue:

1.Loss of Structural Information: The original cracks' width and texture are reduced to a single-pixel line during the thinning process. This reduction may cause a discrepancy between the predicted cracks and the ground truth in situations where the crack width or complexity is substantial, which would lower the IoU score.

2.Noise Amplification: Thinning may erroneously preserve noisy regions as part of the skeleton if the original images contain noise or irregularities. This could result in over-detection or misalignment with the actual cracks.

3.Complex Crack Structures: The thinning process may not be able to adequately handle the complexities of complex or branching cracks, as seen in Images 4 and 7, leading to incomplete or broken crack representations, which further reduces the overlap with the ground truth.

5. Conclusion

The project has successfully implemented a crack detection system using traditional image processing techniques. While effective for simple cases, further enhancements are necessary for complex scenarios. Future improvements could involve integrating deep learning techniques to enhance detection accuracy and robustness.

One notable challenge we encountered was the system's tendency to detect false positives in textures resembling cracks but lacking structural or morphological characteristics. This highlights the need for additional development, potentially through the use of deep learning classifiers capable of distinguishing genuine cracks from background noise. Nonetheless, traditional image processing techniques show promise in delivering favorable results.

6. References

-GIMP (Image manipulation software): <https://www.gimp.org>

- CVAT (Computer Vision Annotation Tool):<https://www.cvat.ai>
- SVMs, Decision Trees: Based on course materials.